ReliefPlan AI Project Submission

Emergency Resource Allocation and Decision Support System

Code | Output | Implementation steps

Submission date 16 October 2026

Project abstract

ReliefPlan AI converts disaster damage surveys, population data and regional stockpiles into severity predictions, area priorities and resource allocation recommendations. The project combines a trained XGBoost classifier with a SciPy linear programming optimizer, then presents results through React and Streamlit interfaces. Human emergency managers review the recommendations and make final decisions.

This submission explains the implemented data flow, setup procedure, mathematical rules and main source code. It also presents results from executing the supplied code and trained artifacts, including area allocations, resource coverage, supply shock simulation, model evaluation records and automated tests.

Verified outcome

The existing test suite passed all 24 tests. The React production build also completed. With the default three day core scenario, 12,846 residents are estimated to need assistance across ten areas. The optimizer allocated 15,000 food rations, 20,000 litres of water and 1,927 medical kits. Water coverage fell from 17.30 percent to 12.11 percent after a 30 percent water supply reduction.

The current system is an educational prototype using synthetic data. Its measured output and implementation limits are explained in this report so that the submission can be evaluated accurately.

Submission files

ReliefPlan_AI_Project_Submission.docx contains the report and selected source listings.
ReliefPlan_AI_Code_and_Output.zip contains the full project source, input data, trained models, execution evidence and reproduction instructions.

Report guide and project objectives

The report is organized so an examiner can understand the project, reproduce its results and inspect the code behind the output.

Section	Coverage
Project design	Architecture, technology and file organization
Input data	CSV inventory, features and training data
Implementation	Environment setup and processing stages
Mathematical rules	Affected population, demand, priority and optimization
Program output	Baseline allocation, scenario comparisons and model results
Verification	Tests, API handler execution and frontend build
Limits and demonstration	Current gaps, examiner walkthrough and submission checklist
Code appendix	Selected implementation listings with file and line references

Problem statement

Disaster response requires decisions about how limited food, water and medical supplies should be distributed among communities with different levels of damage and need. Manual allocation can obscure the assumptions behind these decisions and make scenario comparison difficult.

Objectives

- Predict damage severity from structural damage and disaster features.
- Estimate affected population and resource demand for a chosen response duration.
- Rank areas with a transparent weighted priority score.
- Allocate resources within available stock and area demand limits.
- Explain predictions and priorities, compare supply or population shocks, and record decisions.

Operating scope

The default core analysis uses all ten study areas A to J. The React API analysis filters the data to seven areas A to G. These two paths use different population inputs and consumption assumptions, so their output is reported separately.

System architecture and source organization

The application separates computation from presentation. Both interfaces call reusable functions in core. The React interface sends JSON requests to FastAPI, while Streamlit calls the pipeline from Python.

```
CSV data and saved models
          |
          v
Load and validate -> Merge features -> Predict severity
          |
          v
Estimate affected population -> Calculate demand -> Score priorities
          |
          v
Solve allocation LP -> Build actions and warnings -> Summarize results
          |
          v
React through FastAPI or Streamlit -> Human review -> CSV audit history
```

Path	Implemented responsibility
api.py	FastAPI request models and health, analysis, simulation, metrics and audit handlers
core/data.py and core/model.py	CSV validation, feature preparation, training and inference
core/demand.py and core/priority.py	Affected population, resource demand and area scoring
core/optimizer.py	Resource allocation LP, population baseline and transportation LP
core/actions.py and core/explain.py	Action recommendations, warnings, SHAP and score decomposition
core/simulate.py and core/audit.py	Full pipeline, scenario comparison and CSV decision records
frontend/src and app/Home.py	React four step interface and Streamlit guided dashboard
tests and models	Five test modules and saved model artifacts

The src folder contains earlier modules. The app/_pages_archive folder contains earlier Streamlit pages; it is not an active app/pages directory. These files are retained in the source archive, while this report describes the current core pipeline and guided interfaces.

Input datasets and machine learning features

CSV file	Rows	Role
damage.csv	10	Damage observations keyed by area_id and disaster_id
population.csv	10	Coordinates, population, density and vulnerable age groups
disasters.csv	3	Event type, date, magnitude and reported event impacts
resources.csv	4	Resource type and quantity at each warehouse
warehouses.csv	2	Depot identifiers, names and coordinates
roads.csv	22	Connections, distance, travel time and road condition
hospitals.csv	2	Hospital location, available beds and staffing
shelters.csv	2	Shelter capacity and occupancy
scenarios.csv	3	Named example scenarios
historical_disaster_damage.csv	800	Synthetic labelled model training records

All supplied disaster impact and population records are synthetic. The coordinates describe a Houston study area. Hospital and shelter records are available as supporting data; the current run_full_pipeline function does not perform separate hospital bed or shelter assignment.

Feature preparation

Damage is joined to population on area_id and to disaster attributes on disaster_id. Missing damage counts become zero; building damage can be reconstructed from component counts. Missing density and magnitude use dataset medians, with fixed defaults when a median is unavailable.

The six classifier inputs are building_damage, minor_damage, major_damage, destroyed_buildings, population_density and magnitude. The target is damage_severity with LOW, MEDIUM, HIGH and CRITICAL classes. Predicted severity and operational priority are separate fields.

Implementation steps for setup and execution

Step 1 Prepare the Python environment

Open PowerShell in the emergency-resource-allocation directory. Create an isolated environment and install the project requirements. The React API also needs FastAPI and Uvicorn, which are absent from the supplied requirements file.

```
cd D:\Disaster\emergency-resource-allocation
python -m venv .venv
.\.venv\Scripts\Activate.ps1
python -m pip install -r requirements.txt
python -m pip install fastapi uvicorn
```

Step 2 Use the existing model artifacts

The supplied models directory already contains best_model.pkl, damage_model.pkl, label_encoder.pkl, shap_explainer.pkl and metrics.json. These artifacts were used for the verification results in this report. To train again when needed, run the command below; retraining replaces the model and metric artifacts.

```
python core/model.py
```

Step 3 Run the tests

```
python -m pytest tests -v
```

Step 4 Launch the React application

Run the API from the project directory so its relative data and model paths resolve correctly. Keep the API terminal open, then start Vite in a second terminal.

```
python -m uvicorn api:app --host 127.0.0.1 --port 8000

# In a second terminal
cd frontend
npm ci
npm run dev -- --host 127.0.0.1
```

Open the local URL printed by Vite, normally http://127.0.0.1:5173. The frontend sends requests to http://127.0.0.1:8000. A production build can be generated with npm run build.

Step 5 Launch Streamlit when required

```
streamlit run app/Home.py
```

The Streamlit interface normally opens at http://localhost:8501. React and Streamlit provide alternate interfaces to the shared computation modules.

Implementation steps for the processing pipeline

Step 6 Load and validate data

load_all_data reads the configured CSV files and validate_dataset checks required columns. Negative numeric values and empty tables produce warnings. prepare_ml_dataset joins the tables and fills missing feature values.

Step 7 Train or load the severity model

Training uses an 80 percent stratified training set and 20 percent held out set with random_state 42. Logistic Regression, Random Forest and XGBoost are evaluated. The implementation selects the model with the largest held out macro F1 and saves the model, label encoder, SHAP explainer and metrics.

Step 8 Estimate demand

Load the saved model and decode predictions. Map each severity to an affected population ratio. Multiply affected residents by each resource rate and response duration. The API converts duration_hours to days before calling the pipeline.

Step 9 Calculate area priority

Combine structural impact, estimated demand, an initial shortage proxy, demographic medical need and road access obstruction. Normalize the weights and classify the final score into a risk tier.

Step 10 Solve resource allocation

Solve one bounded linear program per resource with SciPy HiGHS. Supply constrains the total allocation and demand constrains each area. Generate a population proportional baseline for comparison.

Step 11 Explain and present results

Generate action cards and shortage or road obstruction warnings. The explain module supports global and local SHAP values and a component breakdown of priority scores. React displays maps, ranked areas, allocations and scenario results.

Step 12 Compare scenarios and log decisions

The simulator reruns the same pipeline after modifying supply or population. The API analysis handler appends a CSV audit record with model, inputs, statuses and aggregate allocations. The core pipeline itself does not write an audit record automatically.

Mathematical rules and configured assumptions

Affected population and resource demand

```
affected_i = round(population_i * severity_ratio_i)
demand_i_resource = affected_i * daily_rate_resource * duration_days
shortage_i_resource = max(0, demand_i_resource - allocation_i_resource)
coverage_resource = min(1, total_supply_resource / total_demand_resource)
```

Severity	Affected population ratio
LOW	0.10
MEDIUM	0.30
HIGH	0.55
CRITICAL	0.85

Parameter	Core default	API analysis default
Duration	3 days	72 hours converted to 3 days
Food	2 rations per person per day	2 rations per person per day
Water	3 litres per person per day	2 litres or 2.5 for Wildfire
Medical kits	0.05 per person per day	0.12 or 0.18 for Wildfire or 0.25 for Earthquake

Priority score

```
priority = 0.25 * normalized_impact + 0.25 * normalized_demand
         + 0.20 * normalized_shortage + 0.15 * normalized_medical_need
         + 0.15 * access_penalty
```

Impact uses building damage and a severity weight. Medical need uses 1.5 times elderly population plus 1.2 times children population. Until total_shortage exists, the shortage component is a proxy equal to half of estimated demand. The first four components use scenario min max scaling; access is already bounded from road status scores OPEN 0, DAMAGED 0.5 and BLOCKED 1.

The priority tiers are LOW below 0.25, MEDIUM from 0.25 to below 0.50, HIGH from 0.50 to below 0.75, and CRITICAL from 0.75. Consumption rates, severity ratios and policy weights are project assumptions.

Optimization formulation and result interpretation

For each resource, the decision variable x_i is the quantity allocated to area i. D_i is demand, S is available supply and p_i is max(priority_score_i, 0.05). The implementation minimizes a negative benefit to express a maximization problem through linprog.

```
Maximize   sum_i (p_i / D_i) * x_i
Subject to sum_i x_i <= S
           0 <= x_i <= D_i

Python objective coefficient  c_i = -(p_i / D_i)
Python solver                 linprog(..., method="highs")
```

An optimal result maximizes priority weighted demand satisfaction. Because the marginal benefit is priority divided by demand, the area with the largest raw priority score is not necessarily served first. A positive priority weight does not guarantee a positive allocation.

Baseline comparison

```
baseline_allocation_i = min(D_i, S * population_i / total_population)
baseline_shortage_i = max(0, D_i - baseline_allocation_i)
reduction = max(0, total_baseline_shortage - total_optimized_shortage)
reduction_percent = 100 * reduction / total_baseline_shortage
```

The default ten area run reports 0.00 percent total shortage reduction against this baseline. With scarce supply, both methods can consume the same available stock while assigning it differently. The seven area API example reports a small aggregate reduction, as shown in its separate output section. These observations do not support a general claim that optimization always reduces total shortage.

Transportation extension

optimize_transportation_lp adds warehouse to area quantities, stock constraints per depot and demand constraints per area. It subtracts a small great circle distance cost from demand satisfaction benefit. It is a separate callable routine and is not invoked by run_full_pipeline. It does not enforce blocked road path constraints.

Units and rounding

Food is reported in rations, water in litres and medical supplies in kits. The core rounds quantities to one decimal place, and the API rounds area values to integers. Combined shortage totals sum unlike units for a program KPI and should not be interpreted as a physical quantity or used to compare different resource types. Resource specific tables are more informative.

Output from the default core scenario

Execution inputs were all ten areas A to J, the saved classifier and encoder, the original CSV stockpiles, default priority parameters, three days of demand and no scenario shocks. These results were captured on 16 October 2026.

Measure	Observed value
Areas analyzed	10
Total resident population	30,260
Estimated affected population	12,846
Critical priority areas	1
High priority areas	1
Food solver status	OPTIMAL
Water solver status	OPTIMAL
Medical kit solver status	OPTIMAL
Baseline shortage reduction	0.00 percent

Resource	Demand	Supply	Allocated	Shortage	Coverage
Food	77,076.0	15,000.0	15,000.0	62,076.0	19.46%
Water	115,614.0	20,000.0	20,000.0	95,614.0	17.30%
Medical_Kit	1,927.0	5,000.0	1,927.0	0.0	100.00%

The medical stockpile exceeds demand, so the optimizer leaves 3,073 kits unused. Food and water stock are fully used and large deficits remain. The solver returned OPTIMAL for each resource in this captured scenario.

Hand checked example for Area A

Area A has 2,368 residents and predicted MEDIUM severity. The affected ratio is 0.30, giving round(2368 × 0.30) = 710 residents. Water demand is 710 × 3 × 3 = 6,390 litres. The allocation is 6,390 litres, leaving zero water shortage for this area.

Evidence files are baseline_summary.json and baseline_allocations.csv in the packaged evidence directory. These retain the unrounded values used to prepare the report tables.

Output by area and recommended actions

Area	Predicted severity	Priority	Risk tier	Affected
E	HIGH	0.7858	CRITICAL	2,552
H	HIGH	0.7074	HIGH	2,417
C	HIGH	0.4386	MEDIUM	1,036
J	HIGH	0.4336	MEDIUM	1,114
I	HIGH	0.4299	MEDIUM	1,073
B	MEDIUM	0.3362	MEDIUM	1,336
D	HIGH	0.3345	MEDIUM	1,575
A	MEDIUM	0.3118	MEDIUM	710
F	HIGH	0.2772	MEDIUM	568
G	LOW	0.2086	LOW	465

Area	Food rations	Water litres	Medical kits	Water shortage
A	4,260.0	6,390.0	106.5	0.0
B	0.0	0.0	200.4	12,024.0
C	4,542.0	4,313.0	155.4	5,011.0
D	0.0	0.0	236.2	14,175.0
E	0.0	0.0	382.8	22,968.0
F	3,408.0	5,112.0	85.2	0.0
G	2,790.0	4,185.0	69.8	0.0
H	0.0	0.0	362.6	21,753.0
I	0.0	0.0	161.0	9,657.0
J	0.0	0.0	167.1	10,026.0

Area E has the highest priority score at 0.7858, but receives zero food and water in this run. The LP objective rewards priority divided by demand and has no minimum service constraint. This is a consequential behavior to discuss during evaluation.

Example generated recommendation

Dispatch 4,313.0 litres of water to Area C. Area C holds MEDIUM priority (score 0.44) with 1,036 affected residents and 116 destroyed structures. Predicted severity is HIGH. Shortage reduced from 9,324.0 L to 5,011.0 L (46.3% unmet need eliminated).

An action card is a recommendation generated from allocation output. It does not send a convoy or verify delivery.

Output from supply and population simulations

The core simulator was rerun with a 30 percent reduction in water stock and, independently, with a 25 percent population increase. Other baseline settings were retained for each experiment.

Resource	Baseline coverage	Water shock coverage
Food	19.46%	19.46%
Water	17.30%	12.11%
Medical_Kit	100.00%	100.00%

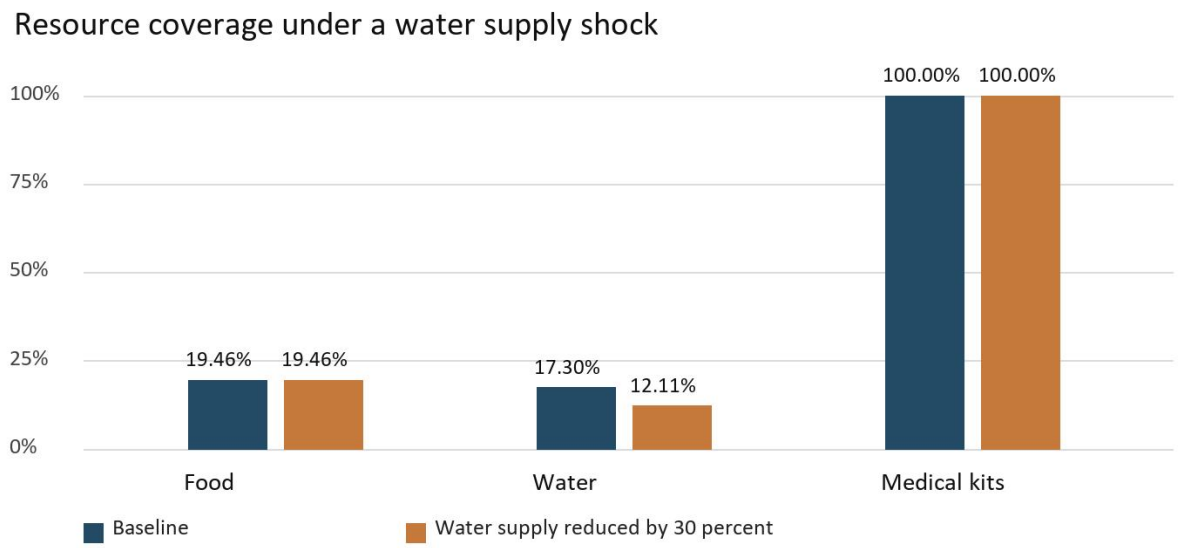


Figure 1 Resource coverage before and after the water supply shock

Measure	Baseline	Water supply reduced
Water stock in litres	20,000	14,000
Water demand in litres	115,614	115,614
Water shortage in litres	95,614	101,614
Estimated affected population	12,846	12,846

The water shock increases unmet water demand by 6,000 litres. In the separate population surge scenario, estimated affected population rises to 16,058; food coverage becomes 15.57 percent and water coverage becomes 13.84 percent.

Evidence files include water_shock_summary.json, population_surge_summary.json and the corresponding allocation CSVs.

Model evaluation output and explainability

The existing models/metrics.json records 800 synthetic training examples, a 640 record training partition and a 160 record held out partition. The saved selection metric is macro F1. This table reproduces the persisted evaluation record; the model was not retrained for this submission.

Model	Accuracy	Macro F1	CV macro F1	ROC AUC
Logistic Regression Baseline	67.50%	0.6765	0.6367	0.9338
Random Forest	78.75%	0.7183	0.6697	0.9464
XGBoost	81.87%	0.7366	0.6634	0.9573

XGBoost is selected with held out macro F1 of 0.7366 and recorded accuracy of 81.87 percent. Random Forest has the highest mean cross validation macro F1 of 0.6697. The implementation selects using the held out metric, so cross validation performance and selection criterion should be distinguished. Accuracy is reproduced at the precision stored in the JSON file.

XGBoost confusion matrix

Rows are actual classes and columns are predicted classes. The class order is CRITICAL, HIGH, LOW and MEDIUM.

Actual class	CRITICAL	HIGH	LOW	MEDIUM
CRITICAL	8	9	0	0
HIGH	4	88	0	7
LOW	0	0	7	4
MEDIUM	0	4	1	28

Executed local SHAP explanation for Area A

Feature	Input value	SHAP value
destroyed_buildings	26	+1.1028
population_density	1,108	+0.3637
minor_damage	69	+0.0935
magnitude	7.5	+0.0578
major_damage	138	-0.0401
building_damage	233	-0.0200

These are contributions to the predicted MEDIUM class score. Positive values support that class and do not establish a physical cause or an ordinal increase in disaster risk. Contributions describe model behaviour, not causation.

API analysis output and interface workflow

The analysis request uses the API defaults: Wildfire, High, 72 hours, seven areas A to G and resident populations totaling 22,455. Rates are 2 food rations, 2.5 litres of water and 0.18 medical kits per affected resident per day. Handler execution used an isolated audit file for the new verification records.


```
{
  "total_affected": 22455,
  "critical_areas": 1,
  "high_areas": 3,
  "solver_status": "OPTIMAL",
  "coverages": {
    "Food": 25,
    "Water": 26,
    "Medical_Kit": 92
  },
  "impact": {
    "baseline_shortage": 102586,
    "optimized_shortage": 101383,
    "reduction_pct": 1
  }
}
```


The total_affected value above is actually the sum of resident populations, not the severity adjusted estimated_affected field. Coverage is calculated from the CSV quantity column. The inputs nominally request 40,000 food, 40,000 water and 2,500 medical kits, but those fields currently write total_quantity rather than quantity and therefore do not change allocation stock.

API simulation output


```
{
  "water_coverage_before": 27,
  "water_coverage_after": 19,
  "coverage_diff_pts": -8,
  "total_shortage_before": 88630,
  "total_shortage_after": 94629,
  "shortage_diff": 5999
}
```


This simulation independently reloads seven areas with CSV populations and core default rates. It does not reuse the analysis screen populations, stock inputs, duration or disaster specific rates. Its rounded 27 to 19 percent water coverage is therefore different from the ten area core experiment.

Four step user workflow

Describe captures disaster details and population inputs. AI analysis displays map markers, priorities and allocations. Response plan shows recommendation cards. What if runs the shock request and displays a bulletin and audit history. Dispatch and bulletin activation are client state demonstrations; external dispatch and messaging services are not connected.

Verification results and execution environment

Check	Observed result
Existing Python tests	24 passed with 1 pandas FutureWarning
React production build	Vite 8.3.2 completed and transformed 1938 modules
Default core pipeline	All three resource solvers returned OPTIMAL
Water supply shock	Coverage decreased and water shortage increased by 6000 litres
Population surge	Pipeline completed and demand increased
Transportation routine	OPTIMAL with 20000 litres shipped from W1
API handler execution	Health, analysis, simulation, metrics and audit handlers returned
Low stock input comparison	Changing API stock inputs to 1 left allocations unchanged

The API checks invoked handlers directly with Pydantic request objects. They verify computation and response construction, rather than HTTP transport or browser interaction. The React check is a production build, rather than an end to end UI test. These checks establish the scope of the evidence.

Test suite coverage

Test module	Tests	Coverage
test_actions_and_explain.py	5	Action quantities, numeric reasons, local SHAP and priority decomposition
test_demand.py	4	Hand checked arithmetic, bounds and zero population
test_optimizer.py	7	Supply and demand limits, solver status, zero supply and transportation
test_priority.py	4	Score bounds, categories, determinism and constant normalization
test_simulate_and_audit.py	4	Water shock, reduction formula, scenario comparison and audit persistence

The warning concerns future pandas behavior when filling missing values in object arrays in core/explain.py. No test failed. Tests do not cover every UI claim or API wiring behavior.

Verified runtime

Python 3.13.7 on Windows 11. Packages: numpy 2.2.6, pandas 2.3.3, scipy 1.16.2, scikit-learn 1.8.0, xgboost 3.2.0, shap 0.50.0, pytest 9.1.1, fastapi 0.135.1, uvicorn 0.41.0, streamlit 1.54.0, plotly 6.5.2, joblib 1.5.3.

Build artifacts were dist/index.html, a 26.60 kB CSS bundle and a 403.65 kB JavaScript bundle. Dependency installation is required after extracting the source archive because node_modules is not packaged.

Current limitations and improvement steps

Connect stock inputs to the optimizer

In api.py, update the quantity field used by core/optimizer.py and distribute requested total stock across warehouse rows without duplicating it. The current total_quantity assignment leaves original CSV supply unchanged.

Implement a feasible minimum allocation policy

The response screen claims a 25 percent survival floor, but LP bounds currently start at zero. Decide how to ration when supply is below the desired floor and encode the chosen policy as constraints before making this claim.

Carry analysis state into simulations

The simulation handler reloads original data and default assumptions. Pass the current populations, duration and stock configuration to make the before and after comparison match the current analysis.

Connect scenario controls to the intended computation

The severity string is used in the analysis audit name, while predicted severity comes from the model. Disaster type changes consumption assumptions but does not replace the damage survey or event magnitude.

Use explicit status and population fields

The API reads a missing singular solver_status key and defaults to OPTIMAL. Return actual solver_statuses and is_optimal. Rename total_affected or compute it from estimated_affected so the KPI matches its label.

Separate prototype actions from operational execution

Dispatch and broadcast controls change React state only. The displayed convoy ETA is a fixed interface value. Real integrations would need delivery confirmation and event records.

Strengthen reproducibility and evaluation

Pin dependencies, select models using training validation rather than the held out test set, and validate with independently collected data. The current training and evaluation records are synthetic.

Strengthen audit and route guarantees

CSV append logging is editable and clear_audit_log can delete it; it is not tamper proof. The transportation LP uses distance penalties, while blocked road information currently produces warnings rather than route feasibility constraints.

Demonstration and submission checklist

Suggested examiner demonstration

1 Introduce the problem

Explain limited stock, different area needs and the role of a human decision maker. Show the source folders and describe the synthetic data.

2 Describe the scenario

Open React and show the seven population fields, duration and disaster type. Explain the current stock input limitation before interpreting coverage.

3 Run and inspect analysis

Run analysis, select a map area and compare predicted severity with operational priority. Show quantities and remaining shortage.

4 Explain the allocation

Use the objective and area table in this document to explain priority weighted demand satisfaction. Show that zero allocations are possible.

5 Stress test water supply

Run a 30 percent water supply shock. Explain that the API simulation uses its independent default baseline; use the packaged core comparison for a controlled ten area experiment.

6 Present validation

Show the 24 passing tests, saved model comparison and SHAP output. End with the current limitations and improvement steps.

Files to submit

- This Word report with implementation explanation, code listings and output tables.
- The complete source and output ZIP, including data, models, tests and frontend source.
- The evidence folder within the ZIP, including JSON output, area CSVs, execution notes and reproduction scripts.

Reproduction of captured evidence

After extracting the ZIP, install the dependencies and run python evidence/capture_outputs.py from the archive root. The script resolves the sibling emergency-resource-allocation directory and writes new verification output to evidence. Its new analysis records use evidence/verification_audit.csv, preserving the project audit history.

The source appendix below contains selected complete Python functions and the React analysis request handler. File paths and source line ranges refer to the submitted project snapshot. All imports, remaining functions, interfaces, style sheets and tests are present in the source archive.

Code listing 1 Data preparation

Source core/data.py | Lines 118 to 159

Exact source function with printed line numbers. Dependencies and surrounding module code are in the source archive.

```
118 def prepare_ml_dataset(datasets: Dict[str, pd.DataFrame]) -> pd.DataFrame:
119     """Merges damage, population, and disaster tables for model training or inference.
120
121     Args:
122         datasets: Dictionary of DataFrames as returned by load_all_data.
123
124     Returns:
125         Consolidated DataFrame with engineered features ready for ML ingestion.
126     """
127     if "damage" not in datasets or "population" not in datasets or "disasters" not in
    datasets:
128         raise ValueError("Cannot prepare ML dataset: 'damage', 'population', or 'disasters'
     missing.")
129
130     # Start from damage table
131     df = datasets["damage"].copy()
132
133     # Join population attributes by area_id
134     pop_df = datasets["population"]
135     df = pd.merge(df, pop_df, on="area_id", how="left")
136
137     # Join disaster magnitude and characteristics by disaster_id
138     disaster_df = datasets["disasters"]
139     df = pd.merge(df, disaster_df, on="disaster_id", how="left")
140
141     # Missing value imputation with safe defaults
142     df["minor_damage"] = df["minor_damage"].fillna(0)
143     df["major_damage"] = df["major_damage"].fillna(0)
144     df["destroyed_buildings"] = df["destroyed_buildings"].fillna(0)
145
146     if "building_damage" not in df.columns or df["building_damage"].isnull().all():
147         df["building_damage"] = df["minor_damage"] + df["major_damage"] +
    df["destroyed_buildings"]
148     else:
149         df["building_damage"] = df["building_damage"].fillna(
150             df["minor_damage"] + df["major_damage"] + df["destroyed_buildings"]
151         )
152
153     median_density = df["population_density"].median() if not
    df["population_density"].dropna().empty else 3000.0
154     df["population_density"] = df["population_density"].fillna(median_density)
155
156     median_mag = df["magnitude"].median() if not df["magnitude"].dropna().empty else 6.0
157     df["magnitude"] = df["magnitude"].fillna(median_mag)
158
159     return df
```

Code listing 2 Affected population estimation

Source core/demand.py | Lines 42 to 59

Exact source function with printed line numbers. Dependencies and surrounding module code are in the source archive.

```
42  def estimate_affected_population(
43      population: float,
44      severity: str,
45      custom_ratios: Optional[Dict[str, float]] = None
46  ) -> int:
47      """Estimates the acute affected population requiring aid in an area.
48
49      Args:
50          population: Total resident population in the area.
51          severity: Assessed or predicted damage severity level.
52          custom_ratios: Optional override for severity affected ratios.
53
54      Returns:
55          Integer count of affected residents.
56      """
57      ratios = custom_ratios or SEVERITY_AFFECTED_RATIOS
58      ratio = ratios.get(str(severity).upper(), 0.30)
59      return int(max(0, round(population * ratio)))
```

Code listing 3 Resource demand calculation

Source core/demand.py | Lines 62 to 100

Exact source function with printed line numbers. Dependencies and surrounding module code are in the source archive.

```
 62  def compute_area_demands(
 63      areas_df: pd.DataFrame,
 64      per_capita_needs: Optional[Dict[str, float]] = None,
 65      duration_days: float = DEFAULT_DURATION_DAYS,
 66      use_affected_only: bool = True
 67  ) -> pd.DataFrame:
 68      """Computes exact demand for each resource type across all areas.
 69
 70      Formula: demand_ik = population_i * per_capita_need_k * duration
 71
 72      Args:
 73          areas_df: DataFrame containing area records with population and severity.
 74          per_capita_needs: Resource consumption standards per person per day.
 75          duration_days: Mission duration in days.
 76          use_affected_only: If True, uses affected population rather than total.
 77
 78      Returns:
 79          DataFrame containing area identifier, population, and demand columns:
 80          'demand_Food', 'demand_Water', 'demand_Medical_Kit'.
 81      """
 82      df = areas_df.copy()
 83      needs = per_capita_needs or DEFAULT_PER_CAPITA_NEEDS
 84      duration = max(0.1, float(duration_days))
 85
 86      # Ensure estimated affected population is computed
 87      if "estimated_affected" not in df.columns:
 88          sev_col = "predicted_severity" if "predicted_severity" in df.columns else
     "damage_severity"
 89          df["estimated_affected"] = df.apply(
 90              lambda r: estimate_affected_population(r.get("population", 1000),
     r.get(sev_col, "MEDIUM")),
 91              axis=1
 92          )
 93
 94      base_pop = df["estimated_affected"] if use_affected_only else df["population"]
 95
 96      for resource, rate in needs.items():
 97          col_name = f"demand_{resource}"
 98          df[col_name] = (base_pop * float(rate) * duration).round(1)
 99
100      return df
```

Code listing 4 Area priority calculation

Source core/priority.py | Lines 119 to 210

Exact source function with printed line numbers. Dependencies and surrounding module code are in the source archive.

```
119 def calculate_area_priorities(
120     areas_df: pd.DataFrame,
121     roads_df: Optional[pd.DataFrame] = None,
122     weights: Optional[Dict[str, float]] = None,
123     thresholds: Optional[Dict[str, float]] = None,
124 ) -> pd.DataFrame:
125     """Calculates normalized priority scores and risk tiers for all areas.
126 
127     Args:
128         areas_df: DataFrame containing area data, damage metrics, population,
129                   and predicted_severity.
130         roads_df: Optional DataFrame of roads for accessibility analysis.
131         weights: Dictionary of component weights.
132         thresholds: Dictionary of risk classification thresholds.
133 
134     Returns:
135         DataFrame enriched with 'priority_score', 'priority_level',
136         and decomposition columns.
137     """
138     df = areas_df.copy()
139     w = weights or DEFAULT_WEIGHTS
140     th = thresholds or DEFAULT_THRESHOLDS
141 
142     # Normalize weights so they sum to 1.0
143     w_sum = sum(w.values()) if sum(w.values()) > 0 else 1.0
144     norm_w = {k: v / w_sum for k, v in w.items()}
145 
146     # 1. Impact: structural damage modulated by predicted severity
147     sev_weights = df["predicted_severity"].map(SEVERITY_WEIGHT_MAP).fillna(0.3)
148     bldg_dmg = df["building_damage"].fillna(0) if "building_damage" in df else pd.Series(0,
     index=df.index)
149     impact_raw = bldg_dmg * sev_weights + sev_weights * 100.0
150 
151     # 2. Predicted Demand: proportional to affected population
152     if "estimated_affected" in df.columns:
153         demand_raw = df["estimated_affected"].fillna(0)
154     elif "population" in df.columns:
155         demand_raw = df["population"] * sev_weights
156     else:
157         demand_raw = pd.Series(100.0, index=df.index)
158 
159     # 3. Shortage: unmet need estimate (if available)
160     if "total_shortage" in df.columns:
161         shortage_raw = df["total_shortage"].fillna(0)
162     else:
163         shortage_raw = demand_raw * 0.5  # Initial baseline proxy
164 
165     # 4. Medical Need: vulnerable demographic groups + injured
166     elderly = df["elderly_population"].fillna(0) if "elderly_population" in df.columns else
     0
167     children = df["children_population"].fillna(0) if "children_population" in df.columns
    else 0
168     medical_raw = elderly * 1.5 + children * 1.2
169 
170     # 5. Accessibility Penalty
171     areas_list = df["area_id"].astype(str).tolist()
172     penalties_dict = compute_accessibility_penalties(areas_list, roads_df)
```

Code listing 4 Area priority calculation continued

Source core/priority.py | Lines 119 to 210

```
    access_raw = df["area_id"].astype(str).map(penalties_dict).fillna(0.0)

    # Normalization inside current scenario
    norm_impact = min_max_normalize(impact_raw)
    norm_demand = min_max_normalize(demand_raw)
    norm_shortage = min_max_normalize(shortage_raw)
    norm_medical = min_max_normalize(medical_raw)
    norm_access = access_raw.clip(0.0, 1.0)  # already bounded [0, 1]

    # Weighted Priority Score
    priority_score = (
        norm_w["impact"] * norm_impact +
        norm_w["demand"] * norm_demand +
        norm_w["shortage"] * norm_shortage +
        norm_w["medical_need"] * norm_medical +
        norm_w["accessibility_penalty"] * norm_access
    ).clip(0.0, 1.0)

    # Risk level classification
    priority_level = priority_score.apply(lambda s: classify_risk_level(s, th))

    # Add results and component breakdowns
    df["priority_score"] = priority_score.round(4)
    df["priority_level"] = priority_level
    df["norm_impact"] = norm_impact.round(4)
    df["norm_demand"] = norm_demand.round(4)
    df["norm_shortage"] = norm_shortage.round(4)
    df["norm_medical_need"] = norm_medical.round(4)
    df["norm_accessibility_penalty"] = norm_access.round(4)

    # Weighted contributions for SHAP-style breakdown
    df["comp_impact"] = (norm_w["impact"] * norm_impact).round(4)
    df["comp_demand"] = (norm_w["demand"] * norm_demand).round(4)
    df["comp_shortage"] = (norm_w["shortage"] * norm_shortage).round(4)
    df["comp_medical_need"] = (norm_w["medical_need"] * norm_medical).round(4)
    df["comp_accessibility"] = (norm_w["accessibility_penalty"] * norm_access).round(4)

    return df
```

Code listing 5 Linear programming allocation

Source core/optimizer.py | Lines 94 to 229

Exact source function with printed line numbers. Dependencies and surrounding module code are in the source archive.

```
 94  def optimize_resource_allocation(
 95      areas_df: pd.DataFrame,
 96      resources_df: pd.DataFrame,
 97      resource_types: Optional[List[str]] = None
 98  ) -> Dict[str, Any]:
 99      """Solves the multi-resource priority-weighted Linear Program.
100
101      Objective:
102          Maximize sum_i (priority_i * x_ik / demand_ik)
103          == Minimize sum_i (-priority_i / demand_ik * x_ik)
104
105      Constraints:
106          sum_i x_ik <= total_supply_k
107          0 <= x_ik <= demand_ik for each area i
108
109      Args:
110          areas_df: DataFrame with 'area_id', 'priority_score', and 'demand_{resource}'.
111          resources_df: DataFrame with 'resource_type' and 'quantity'.
112          resource_types: Optional list of resource types to optimize.
113
114      Returns:
115          Dictionary containing:
116              - 'allocation_df': enriched DataFrame with 'opt_alloc_{res}',
    'opt_shortage_{res}'
117              - 'solver_status': string summary per resource ('OPTIMAL', 'ZERO_SUPPLY', etc.)
118              - 'metrics': objective values, runtime, shortage comparisons
119      """
120      start_time = time.perf_counter()
121      df = areas_df.copy().reset_index(drop=True)
122      n_areas = len(df)
123      r_types = resource_types or list(DEFAULT_PER_CAPITA_NEEDS.keys())
124
125      # Priority weights: ensure strictly positive so even low-risk gets proportional aid
126      if "priority_score" in df.columns:
127          p_weights = np.maximum(df["priority_score"].values.astype(float), 0.05)
128      else:
129          p_weights = np.ones(n_areas, dtype=float)
130
131      solver_statuses: Dict[str, str] = {}
132      is_all_optimal = True
133      total_opt_shortage: float = 0.0
134      total_initial_demand: float = 0.0
135
136      for r_type in r_types:
137          demand_col = f"demand_{r_type}"
138          alloc_col = f"opt_alloc_{r_type}"
139          shortage_col = f"opt_shortage_{r_type}"
140
141          # Get total available supply
142          if "resource_type" in resources_df.columns and "quantity" in resources_df.columns:
143              tot_supply = float(resources_df[resources_df["resource_type"] ==
    r_type]["quantity"].sum())
144          else:
145              tot_supply = 0.0
146
147          if demand_col not in df.columns:
148              df[alloc_col] = 0.0
```

Code listing 5 Linear programming allocation continued

Source core/optimizer.py | Lines 94 to 229

```
            df[shortage_col] = 0.0
            solver_statuses[r_type] = "NO_DEMAND_DATA"
            continue

        demands = df[demand_col].values.astype(float)
        total_initial_demand += float(demands.sum())

        # Handle zero supply case gracefully without solver failure
        if tot_supply <= 0:
            df[alloc_col] = 0.0
            df[shortage_col] = demands
            total_opt_shortage += float(demands.sum())
            solver_statuses[r_type] = "ZERO_SUPPLY (No stockpile available)"
            continue

        # Handle zero total demand case
        if demands.sum() <= 0:
            df[alloc_col] = 0.0
            df[shortage_col] = 0.0
            solver_statuses[r_type] = "ZERO_DEMAND (All areas satisfied)"
            continue

        # Build LP
        # c vector: minimize - (p_i / D_i) * x_i
        c = np.zeros(n_areas, dtype=float)
        for i in range(n_areas):
            if demands[i] > 0:
                c[i] = - (p_weights[i] / demands[i])
            else:
                c[i] = 0.0

        # Inequality constraint: sum_i x_i <= tot_supply
        A_ub = np.ones((1, n_areas), dtype=float)
        b_ub = np.array([tot_supply], dtype=float)

        # Bounds: 0 <= x_i <= demand_i
        bounds = [(0.0, float(demands[i])) for i in range(n_areas)]

        # Solve with SciPy Highs LP solver
        res = linprog(c=c, A_ub=A_ub, b_ub=b_ub, bounds=bounds, method="highs")

        if res.success and res.status == 0:
            solver_statuses[r_type] = "OPTIMAL"
            allocations = np.round(np.clip(res.x, 0.0, demands), 1)
        else:
            is_all_optimal = False
            solver_statuses[r_type] = f"INCOMPLETE ({res.message})"
            # Fallback to bounded proportional if LP failed
            prop = (demands / demands.sum()) * min(tot_supply, demands.sum())
            allocations = np.round(prop, 1)

        df[alloc_col] = allocations
        df[shortage_col] = np.maximum(0.0, demands - allocations).round(1)
        total_opt_shortage += float(df[shortage_col].sum())

    elapsed_ms = round((time.perf_counter() - start_time) * 1000, 2)

```

Code listing 5 Linear programming allocation continued

Source core/optimizer.py | Lines 94 to 229

```
206      # Compute baseline for comparison
207      df = compute_baseline_proportional_allocation(df, resources_df, r_types)
208
209      # Compute comparative performance
210      baseline_shortage_cols = [f"baseline_shortage_{r}" for r in r_types if
     f"baseline_shortage_{r}" in df.columns]
211      total_baseline_shortage = float(df[baseline_shortage_cols].sum().sum()) if
     baseline_shortage_cols else total_initial_demand
212
213      shortage_reduction = max(0.0, total_baseline_shortage - total_opt_shortage)
214      reduction_pct = round((shortage_reduction / total_baseline_shortage) * 100.0, 2) if
     total_baseline_shortage > 0 else 0.0
215
216      return {
217          "allocation_df": df,
218          "is_optimal": is_all_optimal,
219          "solver_statuses": solver_statuses,
220          "execution_time_ms": elapsed_ms,
221          "summary": {
222              "total_initial_demand": round(total_initial_demand, 1),
223              "total_baseline_shortage": round(total_baseline_shortage, 1),
224              "total_optimized_shortage": round(total_opt_shortage, 1),
225              "shortage_reduction_units": round(shortage_reduction, 1),
226              "shortage_reduction_pct": reduction_pct,
227              "overall_status": "OPTIMAL" if is_all_optimal else "PARTIAL_OR_UNCONSTRAINED"
228          }
229      }
```

Code listing 6 Severity prediction

Source core/model.py | Lines 321 to 360

Exact source function with printed line numbers. Dependencies and surrounding module code are in the source archive.

```
321  def predict_damage_severity(
322      model: BaseEstimator,
323      df_features: pd.DataFrame,
324      label_encoder: Optional[LabelEncoder] = None,
325      return_proba: bool = True
326  ) -> Tuple[np.ndarray, Optional[np.ndarray], List[str]]:
327      """Generates severity predictions and probabilities with fallback handling.
328
329      Args:
330          model: Trained classifier model.
331          df_features: Input DataFrame containing required feature columns.
332          label_encoder: Fitted LabelEncoder for class names, if applicable.
333          return_proba: Whether to compute and return class probabilities.
334
335      Returns:
336          Tuple of (predicted_class_labels, probability_matrix_or_None, class_names).
337      """
338      X = df_features[ML_FEATURE_COLS].copy().fillna(0)
339      raw_preds = model.predict(X)
340
341      # Convert numeric predictions to class strings if encoder provided
342      if label_encoder is not None:
343          try:
344              preds = label_encoder.inverse_transform(raw_preds)
345              classes = list(label_encoder.classes_)
346          except Exception:
347              preds = raw_preds
348              classes = list(getattr(model, "classes_", []))
349      else:
350          preds = raw_preds
351          classes = list(getattr(model, "classes_", []))
352
353      proba = None
354      if return_proba and hasattr(model, "predict_proba"):
355          try:
356              proba = model.predict_proba(X)
357          except Exception:
358              proba = None
359
360      return preds, proba, [str(c) for c in classes]
```

Code listing 7 Decision audit record

Source core/audit.py | Lines 28 to 99

Exact source function with printed line numbers. Dependencies and surrounding module code are in the source archive.

```
28  def log_decision_run(
29      scenario_name: str,
30      pipeline_result: Dict[str, Any],
31      model_name: str = "XGBoost",
32      audit_file_path: str = AUDIT_LOG_CSV,
33      additional_notes: str = ""
34  ) -> Dict[str, Any]:
35      """Records a complete execution run into the persistent audit trail.
36
37      Args:
38          scenario_name: Descriptive name of the disaster scenario.
39          pipeline_result: Output dictionary from run_full_pipeline or
    optimize_resource_allocation.
40          model_name: Name of the active ML model.
41          audit_file_path: Path to the audit log CSV.
42          additional_notes: Operational comments or operator ID.
43
44      Returns:
45          Dictionary containing the logged record.
46      """
47      os.makedirs(os.path.dirname(audit_file_path), exist_ok=True)
48      now_utc = datetime.now(timezone.utc).strftime("%Y-%m-%d %H:%M:%S UTC")
49      run_id = f"RUN_{uuid.uuid4().hex[:8].upper()}"
50
51      alloc_df = pipeline_result["allocation_df"]
52      kpis = pipeline_result.get("kpis", {})
53      solver_statuses = pipeline_result.get("solver_statuses", {})
54      params = pipeline_result.get("parameters", {})
55
56      # Priority scores summary
57      p_scores = alloc_df["priority_score"].tolist() if "priority_score" in alloc_df else []
58      avg_priority = round(float(pd.Series(p_scores).mean()), 3) if p_scores else 0.0
59
60      # Severity prediction distribution
61      sev_counts = alloc_df["predicted_severity"].value_counts().to_dict() \
62          if "predicted_severity" in alloc_df else {}
63
64      # Allocation breakdown by resource
65      alloc_summary = {}
66      for col in alloc_df.columns:
67          if col.startswith("opt_alloc_"):
68              r_type = col.replace("opt_alloc_", "")
69              alloc_summary[r_type] = round(float(alloc_df[col].sum()), 1)
70
71      # Status summary
72      all_opt = pipeline_result.get("is_optimal", True)
73      status_str = "OPTIMAL" if all_opt else "SUB_OPTIMAL"
74
75      record: Dict[str, Any] = {
76          "run_id": run_id,
77          "timestamp": now_utc,
78          "scenario_name": scenario_name,
79          "model_name": model_name,
80          "solver_status": status_str,
81          "total_affected_pop": kpis.get("total_affected_population",
    int(alloc_df.get("estimated_affected", pd.Series(0)).sum())),
82          "critical_areas": kpis.get("critical_areas_count",
```

Code listing 7 Decision audit record continued

Source core/audit.py | Lines 28 to 99

```
    int((alloc_df.get("priority_level", pd.Series()) == "CRITICAL").sum())),
        "avg_priority_score": avg_priority,
        "total_shortage": kpis.get("total_shortage_units", 0.0),
        "shortage_reduction_pct": kpis.get("shortage_reduction_pct", 0.0),
        "dispatched_allocations": json.dumps(alloc_summary),
        "predictions_distribution": json.dumps(sev_counts),
        "solver_statuses_detail": json.dumps(solver_statuses),
        "scenario_inputs": json.dumps(params),
        "notes": additional_notes
    }

    df_new = pd.DataFrame([record])
    if os.path.exists(audit_file_path):
        df_new.to_csv(audit_file_path, mode="a", header=False, index=False)
    else:
        df_new.to_csv(audit_file_path, mode="w", header=True, index=False)

    return record
```

Code listing 8 React analysis request

Source frontend/src/App.jsx | Lines 115 to 157

Exact source function with printed line numbers. Dependencies and surrounding module code are in the source archive.

```
115    const runAnalysis = async (navigateToStep2 = false) => {
116      setLoading(true);
117      setError(null);
118      try {
119        // Ensure all populations are clean integers
120        const sanitizedPops = {};
121        Object.entries(areaPops).forEach(([k, v]) => {
122          sanitizedPops[k] = Math.max(1, parseInt(v, 10) || 100);
123        });
124
125        const res = await fetch(`${API_BASE}/api/analyze`, {
126          method: 'POST',
127          headers: { 'Content-Type': 'application/json' },
128          body: JSON.stringify({
129            disaster_type: disasterType,
130            severity: severity,
131            duration_hours: durationHours,
132            food_supply: Number(foodSupply),
133            water_supply: Number(waterSupply),
134            med_supply: Number(medSupply),
135            area_populations: sanitizedPops
136          })
137        });
138
139        if (!res.ok) {
140          const errJson = await res.json().catch(() => ({}));
141          throw new Error(errJson.detail || "Failed to execute AI optimization pipeline");
142        }
143
144        const json = await res.json();
145        setAnalysisData(json);
146        setHasUnsavedChanges(false);
147
148        if (navigateToStep2) {
149          setActiveStep("2 AI analysis");
150        }
151      } catch (err) {
152        console.error(err);
153        setError(`Backend Error: ${err.message}. Ensure FastAPI server is running on port
     8000.`);
154      } finally {
155        setLoading(false);
156      }
157    };
```